

Research brief — Chronostream : la troisième couche temporelle

Destinataire : agent d'implémentation, après validation utilisateur. **Auteur** : Claude (4.7) en mode recherche pour Cipher. **Date** : 2026-05-01 (rev. 2). **Statut** : exploration, à valider avant tout code. **Préambule** : ce document fait suite à `RESEARCH_THIRD_MESSAGE_LAYER.md` (2026-04-24) et à la rev. 1 de `RESEARCH_TEMPORAL_TRINITY.md` . Il restreint le périmètre à **Chronostream uniquement**, validé par l'utilisateur comme la 3e couche choisie. Les alternatives (Dead Man's Switch, Quorum, Meta-burn) sont reportées à des phases ultérieures de la roadmap (§11).

1. Pourquoi Chronostream complète la trinité temporelle

Cipher a aujourd'hui deux couches qui modélisent les **bornes** de la lisibilité d'un message :

Couche	Posture temporelle	Topologie	Direction	Métaphore
Timelock	Différer l'apparition	Point d'arrivée (gate ouverte à t_1)	Vers le futur	<i>Le message se réveille</i>
Burn	Précipiter la disparition	Point de départ (gate fermée à t_2)	Vers le passé	<i>Le message s'efface</i>

Géométriquement, ces deux couches définissent deux **instants** sur la ligne du temps : le moment où le message *devient* lisible (t_1), et le moment où il cesse de l'être (t_2). L'intervalle $[t_1, t_2]$ entre les deux est aujourd'hui un *trou noir d'expérience* : pendant cet intervalle, le message est simplement « là », statique, indifférencié.

Chronostream comble exactement ce vide :

Couche	Topologie	Mode temporel
Timelock	Point	<i>Inception</i> — le message <i>naît</i> à un instant
Burn	Point	<i>Termination</i> — le message <i>meurt</i> à un instant
Chronostream	Intervalle	<i>Extension</i> — le message <i>dure</i> sur un intervalle

Cette triade complète l'axe temporel : **point d'arrivée, intervalle d'existence, point de départ**. Chaque couche est un *mode d'être temporel* irréductible aux deux autres.

Observation importante : Chronostream avec $n = 1$ se réduit exactement à Timelock. Donc Chronostream est techniquement une *généralisation* de Timelock, pas un sibling. Cela ne supprime pas Timelock côté UX (la simplicité conceptuelle reste précieuse), mais cela signifie qu'à terme, le code peut converger : un seul moteur `chrono` qui sert les deux UI.

2. Cartographie des modes temporels — pour mémoire

Sept modes ont été évalués comme candidats pour la 3e couche. Six ont été éliminés :

- **Anchorage / time-capsule** — élégant mais ne porte pas sur la lisibilité du destinataire ; c'est de la *preuve d'existence* pour des tiers. Catégorie différente, à classer à part.
- **Cyclicité / window rythmique** — composable comme une succession de paires Timelock + Burn. Pas un nouvel axe.
- **Causalité / event-bound** — exigerait un oracle externe corrompible. Contrarie l'invariant Cipher.
- **Drift / décroissance** — sous-cas de Chronostream inversé (release de chunks de destruction).
- **Echo / replay** — composable comme plusieurs instances de Chronostream.
- **Dead Man's Switch** — trinité valable mais d'axe différent (*modalités d'agentivité* : présence/absence) ; reportée à la Phase 3 mobile où le heartbeat est plus naturel.

Chronostream est retenu comme la complétion **géométriquement pure** de l'axe temporel.

3. Définition formelle de Chronostream

3.1 Principe

Un message Chronostream est un message **fractionné en n chunks**, chaque chunk étant chiffré vers un round drand distinct $r_1 < r_2 < \dots < r_n$. Le destinataire voit un message-conteneur progressif : à t_k , les chunks 1..k sont lisibles, k+1..n restent opaques.

3.2 Mécanique cryptographique

Réutilise intégralement `tlock-js` + drand quicknet déjà en place. **Aucune nouvelle primitive crypto.**

```
plaintext = [chunk1, chunk2, ..., chunkn]
keys      = [k1, k2, ..., kn]                // clés symétriques aléatoires
ciphertexts = [enc(ki, chunki) for i in 1..n]
sealed_keys = [tlock(ki, roundi) for i in 1..n] // chaque clé scellée vers son r

envelope = {
  chunks: ciphertexts,
  schedule: [round1, ..., roundn],
  sealed_keys: sealed_keys,
  metadata: { startsAt, endsAt, pattern, retentionHorizon }
}
```

L'enveloppe entière est ensuite passée au tunnel Double Ratchet existant comme tout autre message Cipher. Le serveur ne voit qu'une enveloppe ratchet opaque ; il ne voit ni les clés, ni le contenu, ni même la cadence sémantique. Les rounds drand cibles sont publics par construction (drand est un beacon public).

L'invariant zero-knowledge est strictement préservé.

3.3 Flow côté client

1. **Sender** : compose le message, choisit n et la durée totale, fractionne le plaintext en chunks, scelle chaque chunk, envoie l'enveloppe.
2. **Serveur** : route l'enveloppe opaque vers le destinataire. Stocke le ciphertext jusqu'à expiration TTL (cf. §5).
3. **Recipient client** : reçoit l'enveloppe (Double Ratchet décrypte la couche externe), inspecte le `schedule`. Pour chaque round déjà passé, fetch la signature drand correspondante et déchiffre le chunk. Pour les rounds futurs, affiche un placeholder avec ETA.
4. **Drand polling** : à mesure que les rounds drand sont publiés, le client déchiffre les chunks suivants automatiquement, sans nouvelle interaction réseau avec le serveur Cipher.

3.4 Composition avec Timelock et Burn

Composition	Effet
	$t_1 = \text{round}_1$; le stream commence à un instant futur, puis se déroule.

Timelock + Chronostream	Redondant si on accepte que Chronostream subsume Timelock (cf. §1), mais conserve son sens UX dans la phase de transition où les deux features coexistent.
Chronostream + Burn (par chunk)	Chaque chunk individuel s'auto-détruit après lecture. Permet « 7 paragraphes, chacun visible 30 secondes après lecture ».
Chronostream + Burn (global)	L'enveloppe entière s'efface une fois le dernier chunk lu, ou après un délai post-stream.
Timelock + Chronostream + Burn	Pleinement composé. Exemple : « débute lundi 14h, dévoile un paragraphe par heure pendant 7h, brûle 24h après le dernier chunk lu ».

Le pattern de stacking introduit par le commit [9666c49](#) accommode déjà N couches indépendantes ; ajouter Chronostream est strictement additif.

4. UX — du sender à la lecture

4.1 Côté sender — composition

Trois niveaux de complexité possibles, par ordre de coût :

1. **Cadence uniforme** — n chunks répartis également entre t_1 et t_2 . Sender choisit deux paramètres : nombre de chunks et durée totale. *Recommandé pour v1.*
2. **Cadence libre** — sender définit chaque round individuellement (« paragraphe 1 dans 1h, paragraphe 2 dans 6h, paragraphe 3 demain à 9h »). Sender choisit n offsets. *Recommandé pour v1.5 selon retour.*
3. **Cadence sémantique** — sender tague chaque chunk (« contexte / action / conclusion ») et compose une dramaturgie. UI de composition de récit. *Reporté indéfiniment — sort du périmètre messagerie.*

Le composer Chronostream s'intègre dans la barre d'outils existante de l'éditeur de message, à côté des toggles Timelock et Burn :

 Timelock [début]
 Chronostream [n=__ , durée=__]
 Burn [délai après lecture]

L'utilisateur peut combiner les trois librement.

4.2 Côté recipient — lecture

Le composant `TlockGate.tsx` existant peut être généralisé en `ChronoGate.tsx` :

- Si $n = 1$ → comportement actuel (Timelock simple, on ne casse rien).
- Si $n > 1$ → vue progressive :
 - Bandeau d'en-tête : « Chronostream — 3/7 dévoilés, prochain dans 14:23 ».
 - Chunks dévoilés : rendus inline, comme du texte normal mais avec un marqueur visuel discret (timestamp d'unlock).
 - Chunks à venir : placeholder avec compte à rebours individuel («  lundi 14h »).

L'indicateur visuel doit clairement distinguer un Chronostream d'un message en plusieurs morceaux ordinaire — sinon le destinataire risque de croire le message complet alors que des chunks suivront.

4.3 Cas d'usage à fort potentiel

- **Annonce différée graduelle** : « voici les 7 décisions du board, une par jour cette semaine ».
- **Récit chronologique** : journal de voyage qui se révèle au rythme du voyage de l'auteur.
- **Pédagogie / cours** : leçons libérées à intervalles, le destinataire ne peut pas sauter d'étapes.
- **Suspense narratif** : fiction interactive, ARG, jeux de rôle par messages.
- **Disclosure légal/contractuel** : clauses libérées progressivement après signature, ou éléments de contexte d'une négociation.
- **Cadeau étalé** : message d'anniversaire fractionné, lettre qui se déroule sur une semaine.
- **Onboarding / accueil** : guide d'arrivée qui se déroule à mesure que le nouveau membre intègre l'équipe.

Aucune messagerie grand public ne propose ça aujourd'hui. C'est un **différenciateur narratif** que ni WhatsApp ni Signal ne peuvent fabriquer sans réécrire leur modèle de message.

4.4 Risques UX et atténuations

Risque	Atténuation
Annulation impossible : la clé est scellée par drand, donc les chunks futurs <i>ne peuvent pas</i> être rappelés.	Confirmation forte avant envoi (« ce message ne pourra plus être annulé »). UI sender équivalente à celle de Timelock. Possibilité de marquer l'enveloppe comme « abandonnée » côté UI, sans pouvoir empêcher la révélation crypto (pseudo-annulation, à expliquer clairement dans le copy).
Notification fatigue : 7 chunks = 7 notifs ?	Mode silencieux par défaut + résumé optionnel (« notification au début et à la fin uniquement », ou « notification uniquement pour le 1er chunk »).
Asymétrie sender/recipient : le sender voit-il les chunks futurs ?	Oui , comme pour les autres couches — le sender a la clé maître via Double Ratchet, on ne se cache pas à soi-même. À documenter clairement.
Lecture ratée : si le destinataire est offline lors de l'unlock d'un chunk, il le récupère plus tard sans perte (tant que l'enveloppe est encore disponible — voir §5).	Comportement par défaut, identique à Timelock.
Drand outage : si le beacon drand cesse, les chunks futurs ne peuvent pas être déchiffrés.	Risque hérité de Timelock, déjà accepté. À documenter dans <code>CIPHER_PRIVACY_GUARANTEES.md</code> .
Confusion avec les messages multi-parties : Chronostream ressemble à 7 messages successifs.	Indicateur visuel fort (icône, bordure, fond légèrement teinté) qui signe « ce message est un Chronostream, attendre la fin avant de répondre ».

5. Synergie avec les TTL serveur — le point critique

C'est le sujet que tu as soulevé. Il est **plus structurel pour Chronostream que pour Timelock**, et il faut le traiter de front.

5.1 Rappel des règles serveur actuelles

D'après `README.md` § *How it works* :

- **Messages livrés** : supprimés serveur 7 jours après *delivery* (configurable). « Delivery »

= le destinataire a récupéré l'enveloppe.

- **Messages non récupérés** : dropped après 30 jours.
- **Pas d'exception pour Timelock** : la règle s'applique uniformément. Un Timelock à 60 jours fonctionne aujourd'hui parce que le client garde l'enveloppe en local après pickup, pas grâce à une règle serveur. Le serveur a déjà supprimé sa copie au jour 7 post-pickup.

5.2 Pourquoi c'est plus délicat pour Chronostream que pour Timelock

Pour un **Timelock simple**, la séquence est :

1. Sender envoie → recipient pickup au jour D .
2. Serveur supprime sa copie à $D + 7$.
3. Recipient unlock (et décrypte une fois pour toutes) au jour $D + 60$, avec le ciphertext local.

Une seule décryption à un seul instant. Le client n'a besoin du serveur que pour le pickup initial.

Pour un **Chronostream long** (disons 30 jours, 30 chunks, 1 par jour), la séquence est :

1. Sender envoie → recipient pickup au jour D .
2. Serveur supprime sa copie à $D + 7$.
3. Recipient unlock chunk 1 à $D + 1$, chunk 2 à $D + 2$, ..., chunk 30 à $D + 30$.

Le client doit conserver l'enveloppe entière en local pendant 30 jours et déchiffrer les chunks au fur et à mesure. **C'est faisable**, mais ça crée plusieurs frictions :

Friction	Sévérité	Détail
Multi-device sync mid-stream	Élevée	Si l'utilisateur ajoute un nouveau device au jour $D + 14$, le serveur n'a plus l'enveloppe. Le nouveau device doit synchroniser depuis un device existant (P2P) ou depuis un backup.
Backup recovery mid-stream	Faible	Le format <code>.cipherbackup</code> inclut déjà tlock-protected envelopes (cf. fix 88eb64a sur <code>unlockBlockHeight</code>). Étendre au schedule + <code>sealed_keys</code> est trivial. Géré.
		Le serveur drop l'enveloppe avant que le destinataire

Recipient offline > 30 jours au pickup	Élevée	la récupère. Le stream entier est perdu , même si seulement le chunk 1 était dans la fenêtre 30j.
Cohérence sender/recipient si sender perd son device au jour D+14	Moyenne	Sender doit aussi avoir gardé l'enveloppe pour pouvoir re-lire ses chunks dévoilés. Ce point existe déjà pour tous les messages Cipher (le sender garde sa Double Ratchet locale).

La friction « recipient offline > 30j » est le **vrai blocage** : un Chronostream à 30 jours n'a aucun sens si le pickup ne se fait pas dans la fenêtre de 30 jours du serveur.

5.3 Quatre options de résolution

Option	Description	Avantage	Coût
A. Cap dur à 7 jours en v1	Refuser à l'UI tout Chronostream dont <code>endsAt - startsAt > 7 days</code> .	Aucune modif serveur, livraison rapide, cas d'usage couverts à 80%.	Limite l'innovation : pas de stream « un message par jour pendant un mois ».
B. Rétention étendue par message	Le serveur lit le <code>schedule</code> du message Chronostream et étend la rétention à <code>last_chunk_round + grace_period</code> (ex. +7j).	Gère les streams longs. Le <code>schedule</code> étant public (rounds drand), aucune fuite de zero-knowledge.	Modification du scheduler serveur. Demande une révision des invariants CI (<code>privacy-invariants.test.ts</code>) pour autoriser un champ <code>retention_until</code> métadonnée.
C. Hybride	Défaut à 7 jours. Sender peut activer un toggle « stream long » qui étend jusqu'à 30j max. UI prévient explicitement (« le destinataire doit se connecter dans les 30j »).	Conserve la simplicité par défaut. Ouvre la porte aux longs streams sans casser les invariants.	Combinaison des coûts A+B, avec UX additionnelle.
D. Pas d'extension	Garder 7j strict, documenter que les		UX hostile pour les longs

serveur, doc agressive	streams >7j requièrent backup multi-device et exposent au risque drop si le destinataire est offline.	Zéro modif serveur.	streams. Casse la promesse silencieuse de la feature.
---------------------------------------	---	---------------------	---

5.4 Recommandation

Pour la v1 : option A (cap à 7 jours).

Justifications :

1. La très grande majorité des cas d'usage utiles tient en 7 jours (annonces hebdo, cours sur 7 leçons, suspense narratif sur une semaine, journal de voyage d'un week-end prolongé).
2. Évite toute modification du backend et des invariants CI — la livraison est purement frontend + crypto, aligné sur les contraintes du brief précédent (« pas de breaking change sur le format de message »).
3. Permet de valider le concept produit avant d'investir dans une rétention serveur sur-mesure.
4. Conserve la propriété forte de Cipher : « le serveur sait le moins possible, et oublie vite ».

Pour la v2 (si la demande émerge) : option C (hybride).

Si après la v1 les utilisateurs réclament des streams longs (mois entier, trimestre, etc.), introduire le toggle « stream long » avec rétention étendue. La métadonnée `retention_until` reste publique sans révéler le contenu — le serveur sait juste « cet objet doit être gardé jusqu'à T », ce qui est un dérivé direct du `schedule drand` (déjà public).

Note sur l'invariant CI : `apps/bridge/src/__tests__/privacy-invariants.test.ts` enforce que le serveur ne stocke pas certains champs sensibles. Si on ajoute `retention_until`, il faut :

- L'ajouter explicitement à la whitelist des métadonnées autorisées.
- Documenter dans `CIPHER_PRIVACY_GUARANTEES.md` que cette métadonnée existe et ce qu'elle révèle (rien de plus que les rounds drand cibles).

5.5 Et le cap à 30 jours pour pickup ?

Indépendamment de la rétention post-pickup, la règle « drop après 30j non récupéré » s'applique. Pour Chronostream :

- Si le destinataire est offline au moment de l'envoi, l'enveloppe attend jusqu'à 30j.
- Si l'enveloppe est récupérée dans cette fenêtre, le compteur de 7j (ou retention étendue) démarre.
- Si non, le stream entier est perdu — y compris des chunks dont les rounds drand seraient déjà passés.

Recommandation v1 : ne pas toucher cette règle. Documenter clairement qu'un Chronostream est livré à pickup, comme tout autre message. Si le destinataire ne se connecte pas dans les 30j, le message est perdu — comportement standard Cipher, cohérent.

Une mitigation possible (v2+) serait d'autoriser un Chronostream à **commencer son décompte au pickup réel** plutôt qu'à un instant absolu (« 7 chunks dévoilés sur 7 jours à partir du pickup »), ce qui rendrait la feature robuste aux retards de connexion. Mais c'est une variante sémantique forte qui mérite sa propre discussion (cf. question §7.9).

6. v1 minimale — proposition concrète

Objectif : livrable en 2 sprints, parfaitement composable avec Timelock + Burn, zéro modification serveur.

6.1 Périmètre v1

- Cadence **uniforme** uniquement (sender choisit n et durée totale ; les rounds sont équidistants).
- **Texte uniquement** (pas d'attachments fractionnés en v1 ; reportés v2).
- **Maximum 10 chunks** (au-delà, refus UX).
- **Durée totale capée à 7 jours** (cf. §5.3 option A).
- **Composition complète** avec Timelock (offset de départ) et Burn (extinction post-stream ou par chunk).

6.2 Hors périmètre v1

- Cadence libre par chunk (v1.5 selon retour).
- Tagging sémantique des chunks.
- Pause/reprise par le sender.

- Attachments fractionnés (v2).
- Streams longs > 7 jours avec rétention serveur étendue (v2 si demande).
- Stream démarrant au pickup plutôt qu'à un instant absolu (v2+ si demande).

6.3 API conceptuelle

Côté `lib/e2ee/chronostream` :

```
async function sealChronostream(
  plaintext: string,
  options: {
    chunks: number,           // 2-10
    startsAt?: Date,          // composé avec Timelock si présent (sinon now)
    endsAt: Date,             // dernier chunk ; cap 7j en v1
    burnAfterRead?: number,   // composé avec Burn si présent (par chunk ou globa
  }
): Promise<ChronoEnvelope>

function unsealChronostreamChunk(
  envelope: ChronoEnvelope,
  chunkIndex: number,
  drandRoundSignature: Buffer
): Promise<string> // plaintext du chunk
```

6.4 Schéma DB

Additif, pas de breaking change :

- `messages.chronostream_schedule` : JSONB , nullable, contient les rounds et offsets.
- `messages.chronostream_chunks` : JSONB , nullable, tableau de ciphertexts par chunk.

Si `null` → message normal. Si non-null → flow Chronostream, le client le détecte et instancie `ChronoGate`.

6.5 Effort estimé

Composant	Effort
Crypto (<code>lib/e2ee/chronostream</code>)	Faible — boucle sur <code>tlock-js</code>
Backend (<code>routes/socketEvents</code>)	Faible — format additif, scheduler inchangé

Frontend sender (composer)	Moyen — UI de configuration des chunks
Frontend recipient (ChronoGate)	Moyen — généralisation de TlockGate , barre de progression
Tests	Moyen — cas multi-chunks, ordres partiels, composition avec Burn
i18n (6 locales)	Faible — vocabulaire restreint
Backup round-trip	Faible — extension de types existants

Estimation totale : 2 sprints solo si la généralisation de TlockGate se passe bien, 3 si l'UI sender demande plusieurs itérations.

7. Questions à trancher avant de coder

1. **Cadence v1** : uniforme uniquement (recommandé), ou cadence libre dès la v1 ?
 2. **Granularité minimale** : quelle est la durée minimale entre deux chunks ? 3s = round drand minimum, mais UX-wise probablement 1 minute minimum.
 3. **Notifications** : un push par chunk dévoilé, ou un push de début + un push de fin uniquement ? Mode silencieux par défaut ?
 4. **Texte ou multimédia** : v1 texte seul OK ?
 5. **Annulation** : autorise-t-on le sender à *abandonner* un Chronostream en cours (pseudo-annulation côté UI) ?
 6. **Naming** : « Chronostream », « Slow Reveal », « Unfurl », « Unfold », « Distillation », autre ? Le naming porte la narrative, à choisir avec soin avant l'i18n.
 7. **Convergence avec Timelock** : à terme, voit-on Chronostream comme une *généralisation* de Timelock (un seul moteur, deux UI) ou comme deux features côte-à-côte ?
 8. **Cap v1 à 7 jours** : OK pour la v1, ou besoin de streams longs dès le départ ?
 9. **Stream-from-pickup vs stream-from-absolute** : v1 au temps absolu (drand round), ou prévoir l'option « démarre à mon pickup » ?
-

8. Contraintes héritées

(Reprise stricte de §6 et §8 du brief original.)

- **E2EE** : aucune clé en clair côté serveur, jamais.
 - **i18n** : les 6 locales (`apps/frontend/src/locales/`) doivent être à jour dans le commit qui livre la feature.
 - **Backup** : tous les `sealed_keys` et le `schedule` doivent survivre au round-trip export → import. Mettre à jour `apps/frontend/src/lib/backup/types.ts` et `backupService.ts`.
 - **Socket events** : pas d'événement nouveau si évitable. Le client peut polling drand pour récupérer les clés (pattern Timelock existant). Si un event est nécessaire, le protéger par signature Ed25519 comme `call:invite` et `burn_message`.
 - **Pas de breaking change** sur le format de message. Champs JSON additionnels uniquement.
 - **Pas de plaintext serveur**, jamais, sous aucun prétexte.
 - **Privacy invariants CI** (`apps/bridge/src/__tests__/privacy-invariants.test.ts`) : si on ajoute la moindre métadonnée sur `messages`, mettre à jour le test et la doc `CIPHER_PRIVACY_GUARANTEES.md` en conséquence. La v1 cap-à-7j évite ce point.
-

9. Anti-patterns à éviter

- Ne pas réimplémenter tlock — utiliser `tlock-js`.
- Ne pas faire dépendre Chronostream du serveur pour l'ordonnancement des chunks. Le client doit pouvoir les déchiffrer seul, drand est public.
- Ne pas confondre « cadence libre » avec « scripting ». La v1 (et probablement la v2) ne doit pas devenir un mini-langage de programmation de release. Si l'utilisateur veut ça, on en reparle plus tard.
- Ne pas exposer les rounds drand bruts dans l'UI — les traduire en horodatages humains avec timezone locale (le composant `TlockGate` le fait déjà).
- Ne pas créer une 3e feature parallèle à Timelock. Chronostream doit être conçu pour *converger* avec Timelock à terme (cf. question §7.7).
- Ne pas dépasser les 7 jours en v1 sans avoir traité explicitement la rétention serveur (cf. §5).

10. Signaux de « c'est prêt »

- Un sender peut configurer un Chronostream en **3 contrôles maximum** (nombre de chunks, durée totale, contenu).
 - Le recipient voit une **progression claire** avec ETA pour le prochain chunk.
 - La composition Timelock + Chronostream + Burn fonctionne et est testable.
 - Export → import préserve l'état, **y compris pour un stream partiellement dévoilé**.
 - Les 6 locales ont les textes.
 - `CLAUDE.md` a une section sous Key Conventions documentant la couche.
 - Les invariants privacy en CI passent toujours (la v1 ne devrait pas les toucher).
-

11. Roadmap suggérée (vue d'ensemble du paysage trinitaire et au-delà)

Étape	Couche	Prérequis	Cas d'usage principal
Maintenant	Chronostream (3e couche)	Aucun	Messages qui durent — narratif, pédagogie, annonces étalées
v1.5 / v2	Chronostream cadence libre + streams longs	v1 livrée + retours	Composition narrative fine, streams mensuels
Phase 2 mobile	Dead Man's Switch	Mobile (heartbeat naturel via app vivante)	Journalisme, testaments, résistance
Phase 4 protocole ouvert	Meta-burn (cf. brief précédent §G)	Protocole ouvert + fédération	Protection de la <i>relation</i> (qui parle à qui)
Quand groupes stables	Quorum N-of-M (cf. brief précédent §A)	Messagerie de groupe livrée	Décisions collectives, secrets partagés

12. Postface — pourquoi cette trinité est *philosophiquement* correcte

Pour mémoire, et parce que ça aide à défendre le choix face à des challengers :

- **Timelock** est l'expression cryptographique du *kairos* — l'instant attendu, le moment opportun. Le message *advient*.
- **Burn** est l'expression cryptographique de la *léthé* — l'oubli, l'extinction nécessaire. Le message *s'efface*.
- **Chronostream** est l'expression cryptographique de la *durée* (au sens de Bergson) — le temps vécu, étendu, qualitatif. Le message *se déploie*.

Les trois ensemble couvrent les trois rapports fondamentaux de l'humain au temps : l'attente, l'oubli, et l'expérience étendue. Aucune messagerie grand public ne fait ça aujourd'hui. C'est cette complétude philosophique — pas seulement technique — qui rend Cipher défendable comme produit de pensée et pas seulement comme outil de communication.

Fin du brief. À valider avant tout code.